Honest Consensus

In the real case, all four nodes acted correctly and voted for the proposed value of 100. Thus, the voting results showed four votes for 100, which met the required quorum of 3, and value 100 was committed. Consensus was achieved because all nodes followed the protocol, voted with signed messages, and met the required quorum. No node dictated the outcome: instead, nodes voted, and the collective agreement (quorum) was reached. Evidence of agreement is the presence of signed votes and meeting the required quorum that the simulator records. The main reason is the quorum requirement: it dictated what majority was needed to select a value, thus ensuring that the outcome was not defined by a single node's vote but the agreement of the collective. This case demonstrates the basic principle of BFT: trust the process and the evidence of consensus rather than individual actors. It also introduces the general idea of the following cases, in which the protocol will be stressed by using nodes that do not follow the protocol, equivocate, impersonate others, or vote as a group.

Byzantine Behaviour

In this case, Node D was being malicious by voting 50 whilst the honest Nodes A, B and C all voted 100. The vote count was 3-for-100 and 1-for-50. Since there were 4 nodes, the required quorum was 3 (the value of 100) and thus the vote was committed. Consensus was successfully achieved because the honest majority outnumbered the faulty node - the protocol did not have to identify, isolate or punish D. This demonstrates the $3f + 1$ rule: with 4 nodes, the system can tolerate 1 Byzantine fault ($f = 1$). The protocol's underlying assumption is that at least 3 of the 4 nodes are honest - if a second node became malicious, this would no longer be the case. The main takeaway is that BFT protocols are based on quorum arithmetic and majority voting rather than trust in any one participant. Correctness arose from the protocol rules and the combined votes of honest nodes, not from any one node's behaviour. This affirms the mental model of trusting the protocol, rather than any one participant.

Equivocation

In this scenario, the dishonest Node D sent three different values to three different nodes: 100, 50, and 75, which all have valid digital signatures. This is equivocation: the node sends out different but valid information to other nodes. It is more serious than a simple wrong vote because each recipient gets a valid, signed message and cannot suspect the sender based on one message. This makes equivocation more dangerous than a simple wrong vote, because each recipient gets a valid, signed message and cannot suspect the sender based on one message. This is why BFT protocols require nodes to exchange messages and build evidence like signed votes and commit certificates. In this exercise we see that cryptographic authenticity is necessary but not sufficient, because a message could be authentic but come from an equivocating node. We cannot rely on a node's word on what value it voted for, and the protocol needs to build evidence across nodes to establish truth.

Cryptographic Evidence and Key Theft

This scenario demonstrated how signatures are used to establish message origin and integrity and what happens if someone gets access to your private key. Normally, each node signs its own vote, and other nodes check the signature to verify that the message was actually sent from the claimed sender and has not been altered. In the case of the key theft attack, the intruder managed to steal Node A's private key and create a forged message with nodeId: A, value: 999, signed with Node A's key. Since the signature was correct, all other nodes would accept the message thinking that it came from node A. Signature verification failed to distinguish between the legitimate node and the attacker. Thus, the BFT protocol is entirely reliant on the security of the links between nodes and the protection of the cryptographic keys. The problem is that consensus protocols are unable to differentiate between legitimate nodes and attackers. If somebody manages to impersonate node A, then the protocol will trust them just as much as the real Node A. The key lesson to learn here is that cryptography is only as strong as the security of the keys. In the real world, it means that the system's integrity relies on key rotation practices, revocation, and cryptographic hardware and monitoring services.

Sybil Attacks and Membership

This scenario was a Sybil attack simulation where attacker launched an attack by creating multiple fake identities to skew the voting results. Initially, there were 4 validators in the system; A, B, C and D. After the attack there were 4 additional nodes in the network which were secretly controlled by the attacker launching the attack, X1, X2, X3 and X4. This means that the attacker had control of 5 nodes out of the 8 nodes in the network. This is significant because it means that more than half of the nodes in the network were controlled by the attacker. The attacker can exert influence over the outcome of any vote being taken within the network. The exercise demonstrates that consensus protocols do not solve the identity problem. The protocol assumes that the initial validator set is legitimate and known in advance but does not decide who is allowed to participate. That is an admission control and governance issue. Without a mechanism for approving participants, an attacker can generate identities and increase their voting power. Blockchain implementations mitigate against this attack by using proof-of-work, proof-of-stake, permissioned membership, and identity verification. The main takeaway point from this exercise is that many practical BFT implementations rely on external admission control and governance mechanisms. If those mechanisms are not appropriately secured, then the general consensus protocol mathematics offer little protection. Consensus and identity need to be secured together.

Quorum Reasoning and Safety

This scenario was designed to tamper with the quorum rule in order to illustrate the importance of the quorum mathematics for the BFT safety. The correct quorum for the 4-node system is 3, but it was changed to 2 in the modified simulator. As a result, votes could be partitioned so that each value (100 and 50) had the same number of votes (2), satisfying the new quorum requirement. The modified simulator thus incorrectly indicated that safety was violated because multiple values could be committed by different quorums simultaneously, which is a conflict. The property that failed was quorum intersection: if two quorums are selected from 4 nodes, each of size 2, it is possible to choose them in such a way that they do not intersect. This would

allow different subsets of nodes to commit conflicting values, thereby violating safety. By contrast, with the correct quorum size of 3, any two quorums would intersect (each would contain at least one of the nodes that were in the other). Therefore, future decisions by a subset of nodes will always agree with past decisions by honest nodes, preventing a conflict. The exercise demonstrated that BFT safety requires careful specification of the quorum rules, such as 3f+1 nodes needed for a quorum in the case of f failures. A wrong specification of the quorum rules, whether accidentally or maliciously introduced, could undermine the safety of the protocol. This illustrates that the choice of the quorum parameters is a security-sensitive operation that should not be made lightly